VSIM BACKEND ↔ BRIDGE INTEGRATION LAYER

Production-Ready LLM Agent Prompt — Build the backend plugs so the completed Android Bridge connects immediately

0. MISSION

You are working on the EXISTING VSIM PWA and EXISTING VSIM BACKEND. Do not rebuild them and do not create a second backend.

Implement the complete production-grade backend integration/application layer that allows a separately developed VSIM Bridge Android application to connect immediately after the Bridge is completed.

The Bridge is a dedicated Android application on a merchant mobile-money phone. It detects provider SMS events and sends normalized transaction events to this backend.

Build real APIs, authentication, provisioning, persistence, validation, idempotency, reconciliation, ledger integration, Redis coordination, realtime events, admin visibility, monitoring, and tests. Never use fake payment confirmations or mock wallet credits.

1. FIRST INSPECT THE EXISTING PROJECT

Inspect the existing backend and PWA before changing code.

Determine backend framework/language/version, ORM, PostgreSQL setup, authentication, wallet/ledger, deposit/top-up, purchase, withdrawal, admin roles, WebSocket/SSE, Redis, deployment, secrets, API conventions, validation, logging, and tests.

Create a gap analysis. Reuse sound existing infrastructure. Do not destroy working code or replace architecture without justification.

2. TARGET ARCHITECTURE

```
VSIM PWA → existing backend → PostgreSQL.
VSIM Bridge Android → secure Bridge API → existing backend → PostgreSQL.
Backend → Redis for cache, locks, rate limits, queues, and realtime coordination where appropriate.
Backend → WebSocket/SSE → PWA + Admin.
```

PostgreSQL is the authoritative financial source of truth. Redis is never the only copy of financial data. Bridge never connects directly to PostgreSQL.

3. BRIDGE MODULE

Add a dedicated Bridge module using the existing project's conventions. Logical components: bridge routes/controller, service, authentication, provisioning, device management, events, validation, heartbeat, reconciliation, idempotency, audit, and health.

Do not create a second financial system. Integrate with the existing wallet/ledger.

4. DEVICE PROVISIONING

```
Implement lifecycle: UNPROVISIONED → PROVISIONING → ACTIVE → DISABLED/REVOKED → DECOMMISSIONED.
```

Admin provisions a Bridge device and binds it to a merchant/provider. Store bridgeDeviceId, merchant, provider, status, app version, credential metadata, heartbeat/sync timestamps, and revocation information.

A newly installed APK must not automatically become trusted. Support activation, disablement, revocation, credential rotation, replacement of lost/stolen devices, and re-registration.

5. BRIDGE AUTHENTICATION

Implement dedicated Bridge authentication separate from customer authentication.

Prefer device-bound cryptographic identity or another secure mechanism appropriate to the existing infrastructure. Do not use one hardcoded API key for all devices.

Support credential rotation and immediate revocation. Every protected Bridge request must verify authentication and authorization, including merchant/provider binding.

Use clear errors such as DEVICE_REVOKED, DEVICE_DISABLED, INVALID_CREDENTIAL, and MERCHANT_NOT_AUTHORIZED.

6. REQUIRED BRIDGE API CONTRACT

Implement equivalent functionality for: POST /api/bridge/register; POST /api/bridge/authenticate; GET /api/bridge/config; POST /api/bridge/heartbeat; POST /api/bridge/events; POST /api/bridge/sync; POST /api/bridge/acknowledge; GET /api/bridge/status.

Adapt paths to existing conventions, but preserve the contract. Document request/response schemas and error codes.

7. TRANSACTION EVENT CONTRACT

Events should contain stable eventId/idempotency identity, bridgeDeviceId, provider, merchantId, transactionReference, transaction type, amount, currency, provider timestamp, device-received timestamp, and minimum required provider metadata.

Never require the Bridge to send a customer wallet ID as proof of ownership. Never allow the Bridge to say 'credit this user'. Backend determines ownership through deposit matching.

Strictly validate schema, amount, currency, timestamp, merchant binding, provider, and transaction reference.

8. IDEMPOTENCY — NON-NEGOTIABLE

Use PostgreSQL-enforced uniqueness for provider transaction identity, at minimum provider + merchant + provider transaction reference as appropriate.

Support event-level idempotency so Bridge retries are safe. Repeated submissions must return an idempotent result and never create another wallet credit.

Financial correctness must not depend solely on Redis or Android.

9. DEPOSIT REQUEST PLUG

Integrate with the existing PWA top-up flow. A top-up creates a unique deposit request with user, expected amount, currency, merchant/provider, status, creation time, and expiry.

PWA displays authorized merchant payment instructions. Bridge reports the provider transaction. Backend deterministically matches it.

Only a valid matched provider transaction creates the wallet ledger credit. Screenshots, user-entered references, copied SMS text, or user claims never independently credit funds.

10. MATCHING ENGINE

Validate merchant/provider, transaction reference, uniqueness, timestamp window, and pending deposit match.

Prevent one provider transaction from crediting multiple users and prevent multiple events from unintentionally crediting one deposit request.

Support MATCHED, UNMATCHED, AMOUNT_MISMATCH, EXPIRED, DUPLICATE, REJECTED, and REVIEW_REQUIRED. Unmatched funds go to reconciliation.

11. FINANCIAL LEDGER

Connect successful Bridge-verified deposits to the existing wallet/ledger. If the current wallet is mutable-only, strengthen it with an auditable ledger.

Use PostgreSQL transactions to atomically record provider transaction, deposit state, ledger entry, and wallet/account state.

Use exact financial numeric types, explicit currency, and never delete financial history.

12. REVERSAL / CHARGEBACK

Implement reconciliation so a later provider reversal can be associated with the original provider transaction and ledger entry.

On confirmed reversal: preserve original transaction, mark settlement state reversed, create a compensating ledger entry, restrict unsupported funds according to policy, flag the account, notify admin, and audit everything.

Do not permanently ban a user solely because of one unexplained reversal. Use verified evidence and configurable risk states. Repeated verified reversals/fraud indicators may cause deposit, spending, or withdrawal restrictions, suspension, or permanent restriction.

13. PURCHASE PROTECTION

Integrate account status, risk restrictions, and available settled funds into existing purchase authorization.

Users must not purchase VSIM products using reversed, restricted, unavailable, or unsupported funds.

Perform reservation/debit atomically to prevent concurrent overspending. Backend makes the final decision.

14. WITHDRAWAL PROTECTION

Ensure withdrawals use authentication, available settled balance, account status, limits, risk controls, and idempotent request handling.

Bridge devices never approve or execute arbitrary withdrawals.

15. REDIS INTEGRATION

Use Redis for rate limiting, short-lived cache, distributed locks where necessary, job/queue coordination, and realtime event coordination.

Redis is not the financial source of truth. If Redis fails, PostgreSQL remains authoritative and no financial data is lost.

Monitor queue depth and Redis failures.

16. REALTIME PWA + ADMIN

After an authoritative backend state change, publish events such as deposit.created, deposit.confirmed, deposit.rejected, deposit.unmatched, deposit.review_required, deposit.reversed, bridge.online, bridge.offline, and bridge.error.

Realtime events are notifications only. On reconnect, clients reload authoritative state from the backend.

Bridge does not communicate directly with PWA.

17. ADMIN BRIDGE MANAGEMENT

Admin must be able to provision, activate, disable, revoke, reauthorize, inspect health, see app version, merchant binding, queue/backlog, Bridge errors, unmatched deposits, mismatches, duplicates, reversals, and suspicious activity.

Use role-based authorization and audit every privileged operation.

18. HEARTBEAT & HEALTH

Implement Bridge heartbeat. Record device, app version, queue size, last event, last sync, provider, merchant binding, and health state.

Expose ONLINE, OFFLINE, DEGRADED, ERROR, DISABLED, and REVOKED states and alert when heartbeats become stale.

19. AUDIT & SECURITY

Audit provisioning, activation, revocation, credential changes, authentication failures, event ingestion, duplicates, matching decisions, ledger-impacting operations, reversals, reconciliation, risk changes, and admin actions.

```
Use correlation IDs linking Bridge event → provider transaction → deposit → ledger → PWA/Admin event.
```

Never log passwords, tokens, private keys, database credentials, or full sensitive SMS.

20. API HARDENING

Strict schemas, authentication, authorization, rate limiting, secure headers/CORS, request-size limits, safe errors, server-side timestamps, replay protection where appropriate, and protected admin endpoints.

Do not expose stack traces or database errors to clients.

21. DATABASE MIGRATIONS

Inspect the existing schema first. Create safe migrations only for missing Bridge/supporting entities.

Do not wipe production data. Add unique constraints and indexes for Bridge IDs, provider references, merchant IDs, statuses, timestamps, and reconciliation.

Use safe foreign keys and never accidentally cascade-delete financial history.

22. TESTS

Create unit, integration, and end-to-end tests for registration, authentication, revoked devices, merchant binding, valid events, duplicates, retries, wrong amounts, unmatched/expired deposits, concurrency, backend outage, Redis outage, realtime notification, reversal, restricted accounts, purchase blocking, admin provisioning, heartbeat timeout, and audit generation.

Prove that one provider transaction can never create two wallet credits.

23. SECURITY TESTING

Test authentication bypass, cross-merchant authorization, IDOR/resource access, replayed events, modified amounts/references, forged bridgeDeviceId, revoked devices, rate-limit abuse, malformed payloads, concurrency, and privilege escalation.

Client-side validation is never a security boundary.

24. DEPLOYMENT & SECRETS

Separate development, staging, and production. Use secure environment/secret management. Never commit secrets.

Secure Node.js behind HTTPS/reverse proxy as appropriate. Restrict PostgreSQL and Redis network access to trusted services.

Never expose database or Redis credentials to Android.

25. BACKWARD COMPATIBILITY

Do not break existing PWA functionality. Preserve existing APIs where possible or version changes.

If no Bridge is online, PWA must show a truthful pending/unavailable state, never fake success.

26. FINAL CONNECTIVITY CONTRACT

When the Android Bridge is finished, it should require only the production API base URL plus approved provisioning/authentication and merchant/device configuration.

It must NOT require PostgreSQL credentials, Redis credentials, manual wallet edits, hardcoded user IDs, or database manipulation.

```
Connection: Bridge → authenticate → heartbeat → send normalized transaction event → receive acknowledgment →
backend validates/matches/records → PWA/Admin receive authoritative update.
```

27. AGENT EXECUTION PLAN

Phase 1 inspect and gap analysis. Phase 2 database/migrations. Phase 3 provisioning/authentication. Phase 4 event ingestion/idempotency. Phase 5 deposit matching/ledger. Phase 6 Redis. Phase 7 realtime. Phase 8 heartbeat/admin. Phase 9 reversal/reconciliation/risk. Phase 10 tests/security/deployment/documentation.

Do not start by building random UI. Implement the integration contract and security-critical infrastructure first.

28. DEFINITION OF DONE

A separately built production Bridge can be provisioned and connected without manual database edits.

```
A real provider event can travel Bridge → API → validation → PostgreSQL → matching → ledger → PWA/Admin.
```

Duplicate/retry cannot duplicate credit. Revoked devices cannot submit. Unmatched/mismatched payments cannot auto-credit. Reversals can be reconciled. Redis failure does not destroy financial truth. Critical operations are audited. Tests pass and the Bridge API contract is documented.

29. REQUIRED OUTPUT FROM YOU

Before coding, provide the gap analysis against the existing repository.

During implementation, identify every file/module changed and explain security-critical decisions.

After implementation, provide API documentation, database migration summary, authentication/provisioning flow, Bridge integration contract, required environment variables, provisioning procedure, test results, security review, remaining blockers, and exact steps for connecting the finished Android Bridge.

Do not claim production-ready if any critical component is mocked, simulated, hardcoded, or unverified.

END-TO-END CONNECTION THAT MUST WORK

Admin provisions Bridge → Bridge authenticates → Bridge sends heartbeat.

PWA creates PENDING deposit → user pays merchant → merchant phone receives SMS.

Bridge parses SMS → stores locally → uploads event → backend authenticates and validates.

Backend checks merchant/provider/reference/idempotency → PostgreSQL persists provider transaction.

Matching service finds the correct pending deposit → financial transaction is committed atomically.

Backend emits deposit.confirmed → PWA and Admin update.

If later reversed → compensating ledger entry + account/risk handling + admin notification.

At no point does Android directly modify PostgreSQL, directly credit a wallet, or choose which user receives money.